

ARCHITECTURE DOCUMENT

System Architecture

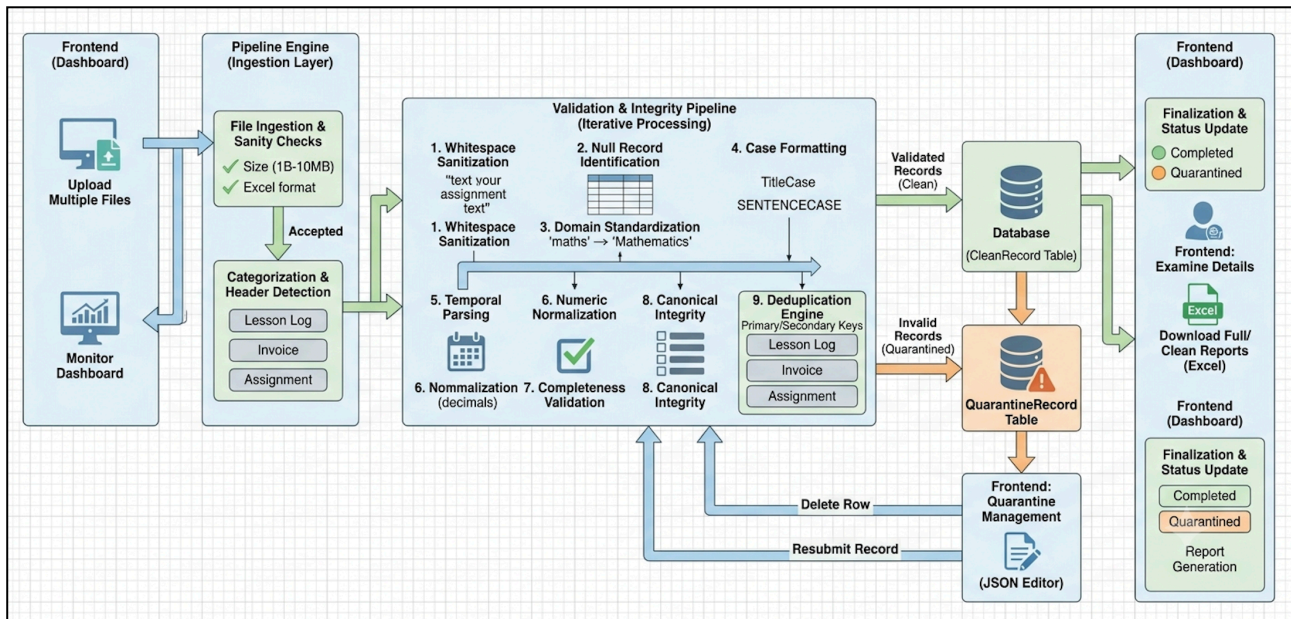


Figure 1: Screenshot of architecture diagram

Users can upload multiple files via the web dashboard, programmatically through the API, or by uploading files in the raw/ directory then run commands on terminal. Upon upload, each file undergoes an initial validation check for file size (must be between 1 byte and 10MB) and must be an Excel file. Files that pass these criteria are then processed by the Header Detection Algorithm to determine their file type.

Once categorised, the data moves through the processing pipeline. Valid records are saved directly to the database, while invalid rows are routed to the quarantine records table with detailed error logs, allowing users to identify precisely where the validation failed. Records that require resubmission will go through pipeline processing again before marking it as correct.

The final upload status is marked as completed (if all records are valid) or quarantined (if any records failed validation). Users can view the full processing report on the website and download either the full processing report or only the clean, validated records.

Pipeline Flow

From the moment the file is uploaded until the processing report is generated, it follows this lifecycle:

The system will extract all uploaded files from the request. It will validate the file based on its size and extension. If either the file size is 0 bytes, exceeds 10MB or file is not in excel format. The pipeline will stop. Next, use the header detection algorithm which will be stated in next chapter to determine the type (lesson_log, invoice, or assignment. If the headers are not detected or the category is unknown, the status is set to unknown and the process stops. Then, proceed to process the data row by row to undergo standardisation and validation which will be stated in next few chapters. For valid rows will be committed to the database as clean records. Invalid rows are diverted to the QuarantineRecord table with an error code and reason details to see exactly what failed without stopping the entire process. Once all rows are processed, the Upload status is updated to completed (if 0 errors) or quarantined (if any rows failed). The processing report is produced.

Header Detection Algorithm

The header detection algorithm will be the first step in the pipeline, as the data needs to be categorised under one of the known categories (assignment, lesson log, or invoice) before the system knows how to process it.

Firstly, check if a header exists in the data by using visual formatting clues, such as bold text, coloured backgrounds, and unique column values. If the header is not in the first row but appears within the first 30 rows, we must skip all rows before the header.

Next, regardless of headers found or not found. It will go through the current stage as the current header may not fit as well. For example, in the Invoice dataset, a column containing values starting with "TAS" might be incorrectly identified as a "Tutor ID" based on the primary scan, when it should be classified as an "Assignment ID". This secondary pass allows the pipeline to re-evaluate columns based on the broader context of the file, ensuring that labels are assigned correctly.

The identification column headers will be using a two-pass heuristic approach:

Pass 1 (Anchor Identification): Searches for predefined, domain-specific strings (e.g., "Log ID", "@tutors.com", status enums). If a cell matches, it flags that row index as a potential anchor.

Pass 2 (Contextual Inference): If a column is still unidentified, it attempts to "fit" the data into known types:

- Dates: Checks if a field is date first as the parsing decimal function can also extract digits from date fields.
- Money/Numbers: If is not a date, then attempts to parse into decimal.
- Specific Patterns: Uses key word and key phrases such as session_note_words (e.g., "reading", "covered") to guess text-heavy columns.

Once header is identified. It will stopped and proceed to next column until all columns are scanned. For fields that not matched with the rules will be replaced with a name starts with "Column_" and their column index.

Inputs that may return the wrong results would be:

- 1) Non-Unique Headers: The current algorithm assumes that a header row must contain unique values. If a header row contains repeated column values, the system may fail to detect the header.
- 2) Assumptions: When two date fields are present, the system defaults to assuming the 'Invoice Date' precedes the 'Payment Date,' which may lead to incorrect labelling in cases of pre-payments before the invoice date.
- 3) Rule-Matching: As the identification rules are hard-coded, any data that does not strictly conform to these patterns will be rejected by the system."

Validation Rule Decision

There are three known file types: assignment, invoice, and lesson log.

1. Strip leading, trailing, and internal excess white whitespaces from all fields.
2. Identify null records which means all data in a row is missing.
3. Apply standardisation for specific text fields. For example, payment status that starts with "pend" should be replaced with "Pending" and subject that starts with "maths" should be replaced with "Mathematics".
4. Apply case formatting to specified text fields. Fields such as student names, tutor names, levels, subjects must be converted to title case to ensure every word is properly capitalised. . Additionally, statuses should be capitalised. Finally, all identifiers—specifically invoice id, assignment id, and invoice id must be converted to uppercase.
5. Perform date parsing for all date fields: invoice date, payment date, session date, and start date. The date needs to be in expected format for it to be convert to iso format.
6. Perform decimal parsing for all float fields: amount, fees charged, duration in hours, and hourly rate. This is to checks for non-numeric value or missing and null values. If it passed the validation the value will remove commas and white spaces then left with digits and round to their specified decimal points.
7. Identify missing fields. Using mapping to state which fields in each file type is required. Fields like payment date and payment notes can be null as student may not made payment for the invoice yet.
8. Identify non-enum values. Values that are not part of the canonical value set.
9. Identify duplicate id and content by creating a dictionary that contain the file type as a key then the value is another dictionary that contains the first key as primary key with the id as its values and the second key as secondary key with the fields that constitute a duplicate content in each file type. Each file type's is unique so it shouldn't be duplicated. For the invoice, a student with same name getting the same invoice date, payment status, and payment date is highly unlikely. For the assignment, a tutor and student with same names for both getting paired on the same date is highly unlikely. For the lesson log, a tutor shouldn't be teaching a student the same chapter for more than a session hence unlikely for it to be duplicated.

Canonical value sets

I have used static canonical sets (the enums and model choices) and dynamic canonical sets (the database models). Both of which ensures data integrity.

1. Static canonical sets (enums and model choices)

The enum classes and model choices are hard-coded business rules that remain constant regardless of user input. Enums is found in the constants.py file and model choices is founded in the model.py file of each Django app. They are used in the header detection and the validation stage during pipeline processing. If a value isn't in these sets, the record is rejected instantly which means it will not be assigned to a header or be flagged during pipeline processing. The row will eventually be placed in quarantined. The static set ensure my data don't include any data other than those i defined.

2. Dynamic canonical sets (database models)

The Tutor, Student, Level, Subject, Assignment, LessonLog, and Invoice models are data-driven canonical sets. Instead of hard-coding every possible entity into an enum, it is being stored in a model. During processing, the pipeline doesn't just check if an entity exists; it uses `get_or_create` to ensure that it only represents that entity. If the record exists, it maps the entity to the existing object. If the record does not exist, it creates a new entry in the table and maps the log to the new ID. The dynamic set helps prevent multiple entries from the same entity.

Schema Design

The ingestion pipeline is designed for full data accountability. Other than saving valid records to the database, tracking why invalid records cannot be saved to the database is also a necessary. To ensure no silent data loss, it categorises every row into two states: cleaned(satisfies all validation), quarantined(did not satisfies validation fully). As such, I have two main tables: one to track uploads and another to track data from the uploads.

The table shown below is used to track uploads.

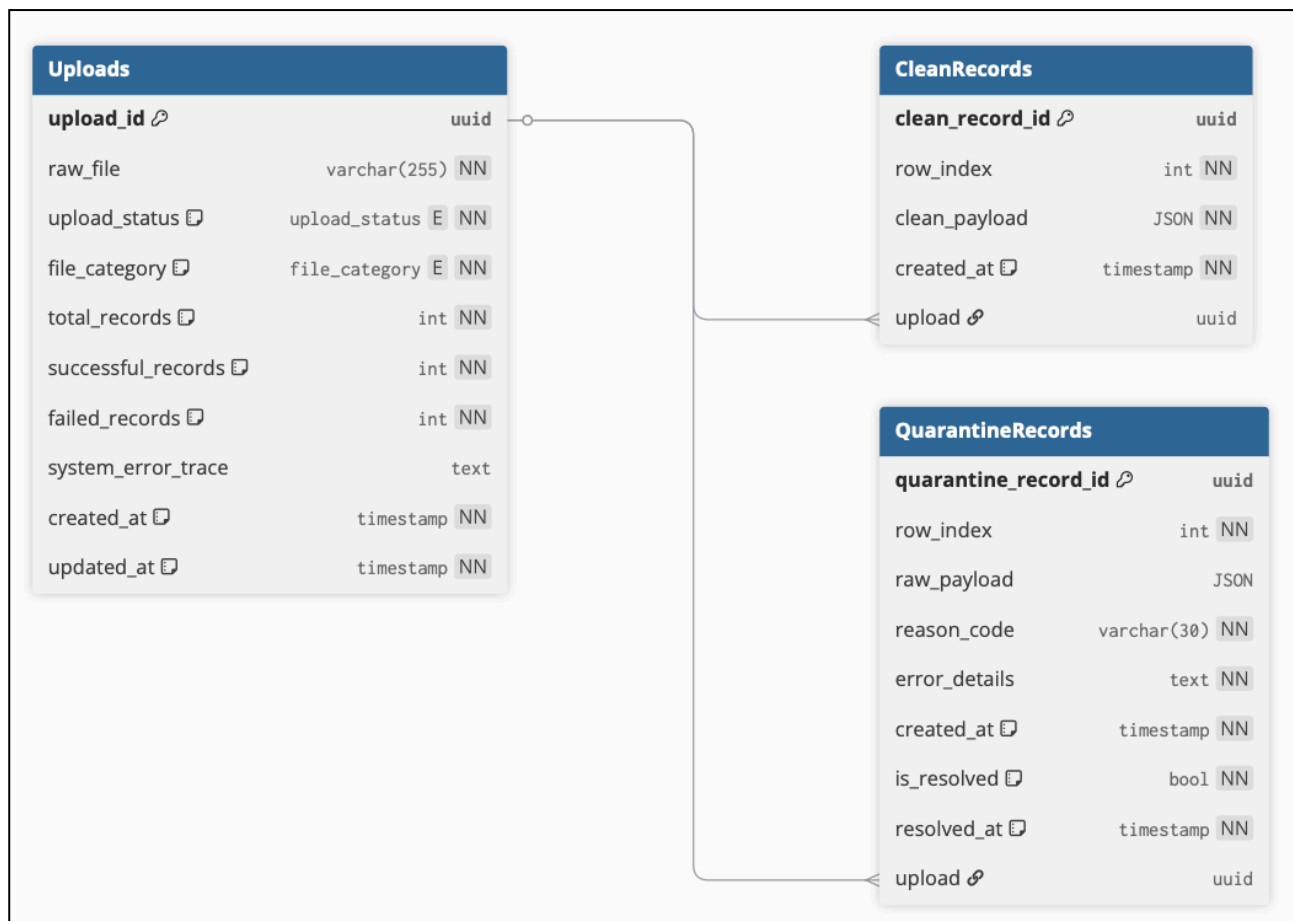


Figure 2: Uploads, CleanRecords, and QuarantineRecords table

PK means primary key. It is the unique identifier for each database record. FK means foreign key. It references primary key from another table.

Uploads table

upload_id(PK): the upload's id

raw_file: the file path to the original file

upload_status: the outcome of pipeline whether is completed with all cleaned records or there is quarantined rows

file_category: the file category (eg. INVOICE, ASSIGNMENT, LESSON_LOG, UNKNOWN)

total_records: the total number of records processed

successful_records: the number of records that passed the validation

failed_records: the number of records that failed the validation
system_error_trace: the reason why the system failed
created_at: the timestamp when the file was first uploaded
updated_at: the timestamp when the file was being updated

It is best practice to keep original source file before being processed. I also keep track why the system might fail so I can fix it.

CleanRecords table

clean_record_id(PK): the clean record's id
row_index: the row index from the original file
clean_payload: the cleaned data being processed
created_at: the timestamp when the record was first created
upload(FK): upload_id(PK) from Upload table - the upload owns the clean record

A upload can own more than one clean record.
A clean record can only be owned by one upload.
This is an one-to-many relationship.

QuarantineRecords table

quarantine_record_id(PK): the quarantine record's id
row_index: the row index from the original file
raw_payload: the actual data being transmitted
reason_code: the category of why it was quarantined
error_details: the context of why it was quarantined
created_at: the timestamp when the record was first created
is_resolved: the boolean whether the record was able to resolve
resolved_at: the timestamp when the record was resolved
upload(FK): upload_id(PK) from Upload table - the upload owns the quarantine record

A upload can own more than one quarantine record.
A quarantine record can only be owned by one upload.
This is an one-to-many relationship.

Next, the table shown below is used to track data from uploads.

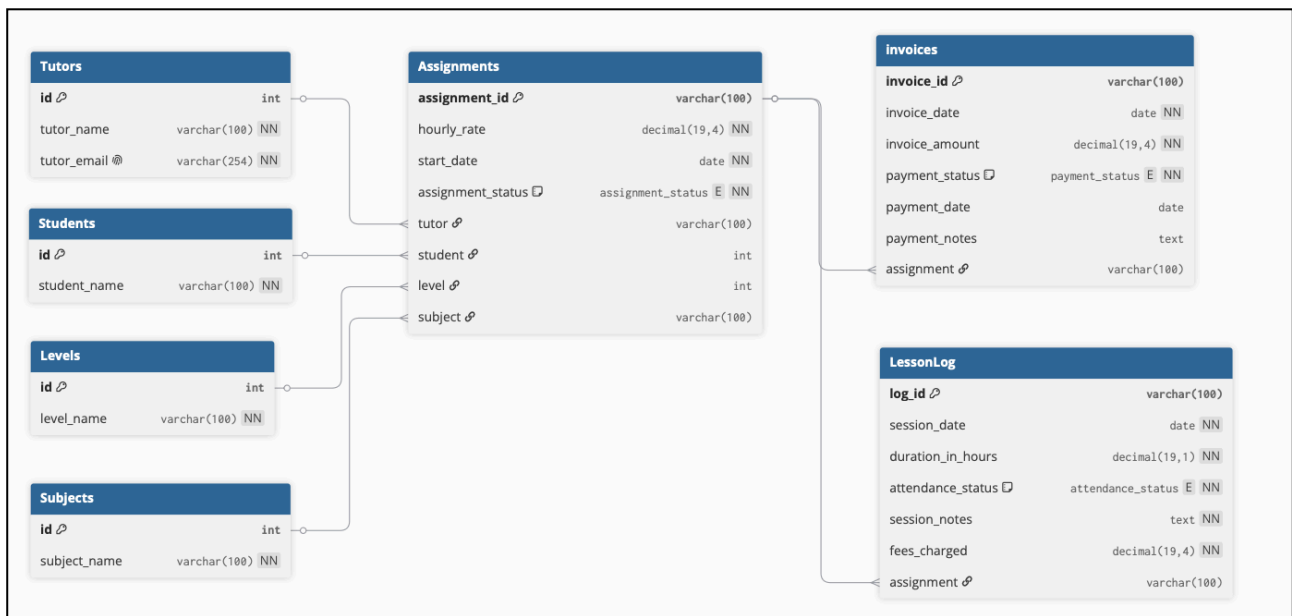


Figure 3: Tutors, Students, Levels, Subjects, and Assignments table

I write database models using Django. The id of tutor, student, level, and subject is automatically generated by Django as the data from the excel files does not contain them.

Tutors table

tutor_id(PK): the tutor's id

tutor_name: the tutor's name

Students table

student_id(PK): the student's id

student_name: the student's name

Levels table

level_id(PK): the levels's id

level_name: the level's name

Subjects table

subject_id(PK): the subject's id

subject_name: the subject's name

Assignments table

assignment_id(PK): the assignment's id

hourly_rate: the tuition fee per hour

start_date: the date the tutor and student got paired

assignment_status: the current phase of the pairing process between a tutor and a student

tutor(FK): id(PK) from Tutor table - the tutor links to the assignment

student(FK): id(PK) from Student table - the student links to the assignment

level(FK): id(PK) from Level table - the level taught during the assignment

subject(FK): id(PK) from Subject table - the subject taught during the assignment

A tutor can link to more than one assignment.
An assignment can only be linked to one tutor.
This is an one-to-many relationship.

A student can link to more than one assignment.
An assignment can only be linked to one student.
This is an one-to-many relationship.

A level can be taught during more than one assignment.
An assignment can only be taught at one level.
This is an one-to-many relationship.

A subject can be taught during more than one assignment.
An assignment can only be taught at one subject.
This is an one-to-many relationship.

Multiple tutors can teach the same student.
The same tutor can teach the same student at different level and subject.

Invoices table

invoice_id(PK): the invoice's id
invoice_date: the date the invoice was sent
invoice_amount: the calculated total fee for the assignment
payment_status: the current stage of a transaction in the billing cycle
payment_date: the date the payment was made
payment_notes: the note for the payment
assignment(FK): assignment_id(PK) from Assignment table - the assignment links to the invoice

An assignment can link to more than one invoice.
An invoice can only be linked to one assignment.
This is an one-to-many relationship.

The payment date and payment details can be null values because students may not pay yet.

Invoices table

invoice_id(PK): the invoice's id
session_date: the date the tutor teaches the student
duration_in_hours: the amount of time in teaching by hour
attendance_status: the attendance status of the student for the session
session_notes: the note for the session
fees_charged: the calculated total fee for the session
assignment(FK): assignment_id(PK) from Assignment table - the assignment links to the invoice

An assignment can link to more than one invoice.
An invoice can only be linked to one assignment.
This is an one-to-many relationship.

Failure behaviour

The database may become unavailable if a file size is too large means it would need more time for processing. A corrupted file means no data or data that would cause all records to be quarantined. In this case, either delete or make some changes. Zero-byte file means that the file is empty. All these will be flagged and the operation team can decide to keep it or not.

Migration log

During the development phase, I manage schema changes by deleting the database, updating the schemas, and re-running migrations. These changes are primarily to address naming inconsistencies and typographical errors. If I need to add new non-null fields, I may delete all database records and re-process the files to ensure integrity, as I want to avoid the 'dirty data' issues associated with applying default values to historical records. Otherwise, I focus on understanding the relationship between the new and old schemas. I have researched setting a non-null field to a nullable one with a default value and then reverting it, but I believe that creates dirty data, which is not suitable for real-life scenarios.

Self-critique

I am least confident in producing structured, appropriate level log entries. This is a skill I have yet to develop, and I am still determining the best practices for when and why to log information. Currently, my logs are cluttered and difficult to interpret. Even when errors are successfully caught, I struggle to understand the output. If I had two more weeks, I would dedicate time to building a structured logging system. This would allow me to quickly identify exactly where a process fails, ultimately saving time on debugging the small, repetitive errors—such as typos or text casing issues which currently consume much of my development time.

Extension: Frontend Dashboard

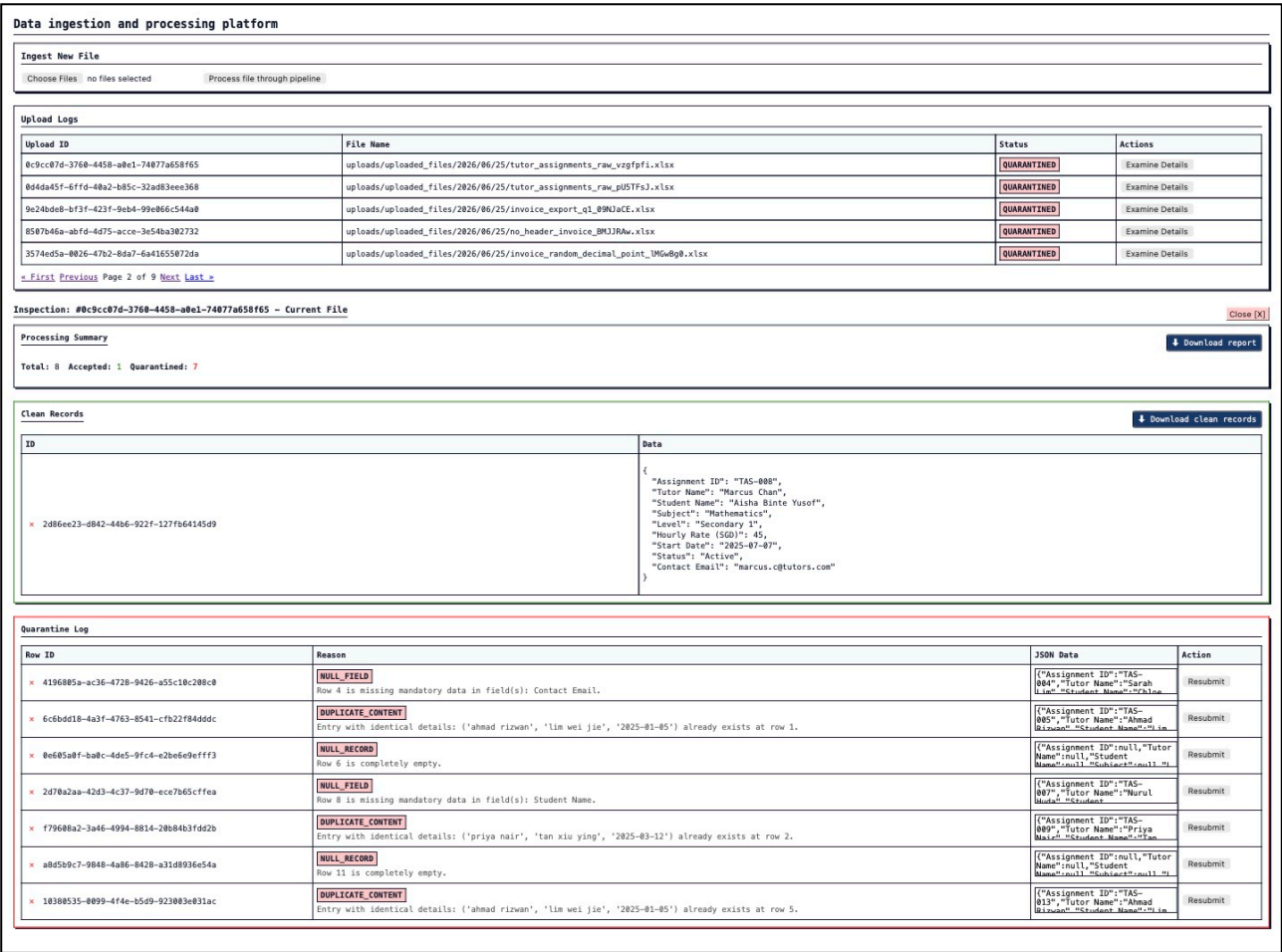


Figure 4: Screenshot of full view of website

The dashboard enables users to upload multiple files and trigger the processing pipeline with a single click on the button. Once processing is complete, users can select "Examine Details" to view a comprehensive summary of clean records and quarantine records. The website allows users to download either the full report or only the clean records in Excel format. Additionally, users can delete rows or resubmitting invalid records by editing them directly in JSON format. Additionally, users can also view database records in Django admin page.

References

1. Hui En, T. (2026) OPUS Tuition [Online] dbdiagram. Available from: <https://dbdiagram.io/d/OPUS-Tuition-6a17dcb6dfb20dafcd0faca3> [Accessed 24 June]